Sisteme inteligente
#12

Principiul de lucru al RNA pentru recunoaşterea formelor.

Principiul de lucru al RNA pentru identificarea sistemelor (aproximarea funcțiilor - probleme de regresie).

(2ore)

RNA multistrat

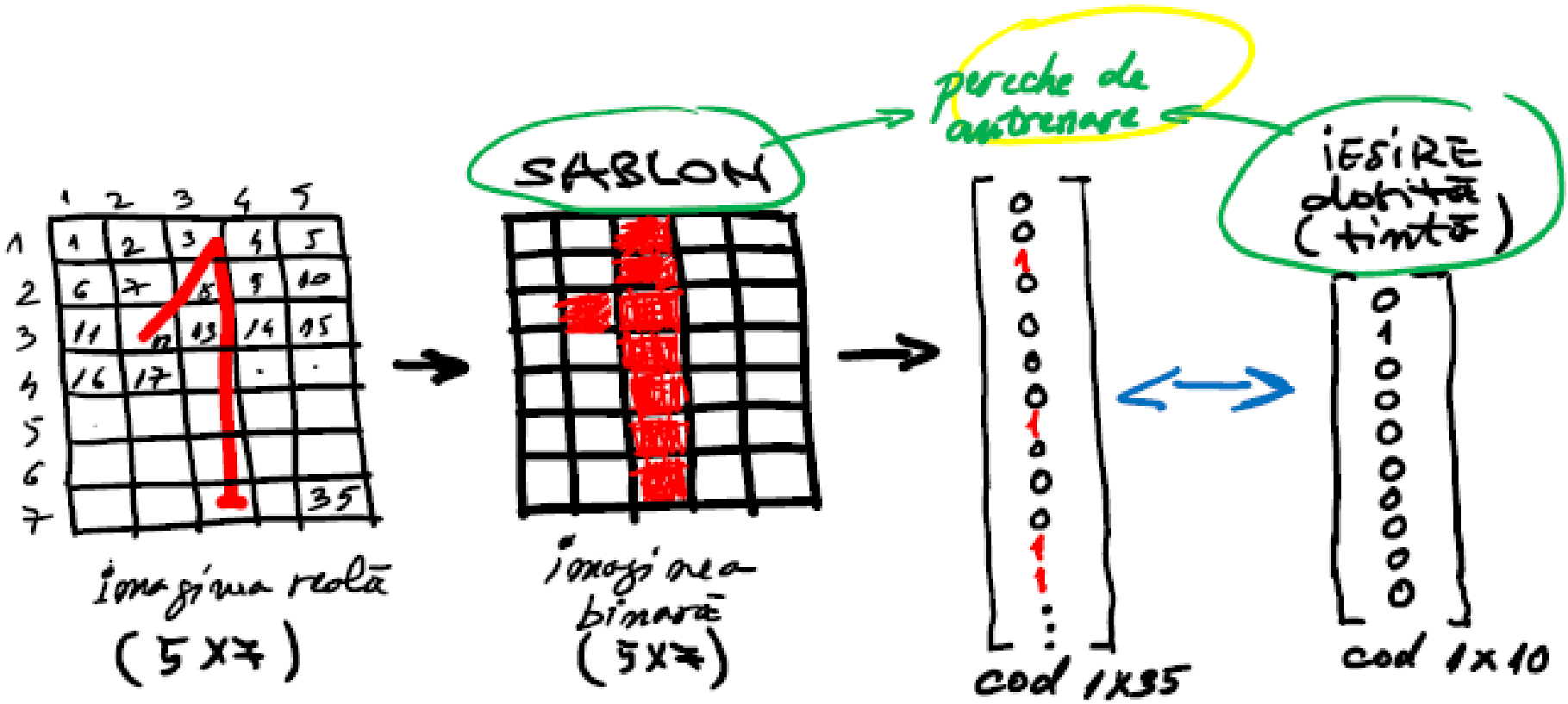
- Modelul RNA de referință: *Perceptronul* – rețea *feedforward* total conectată.
- RNA multistrat poate fi utilizată atât pentru *aproximarea funcțiilor*, cât și pentru problemele de *recunoaștere a formelor* folosind metoda de învățare supervizată.
- Dacă datele prezintă *intervale de întârziere* (sub formă de *serii temporale*), RNA poate fi utilizată *în regim dinamic* și pentru *probleme de predicție*.
- În continuare:
  - se arată **cum se poate utiliza o rețea multistrat**.
  - se prezintă ***metodologia* de bază pentru proiectarea** oricărei rețele neuronale.

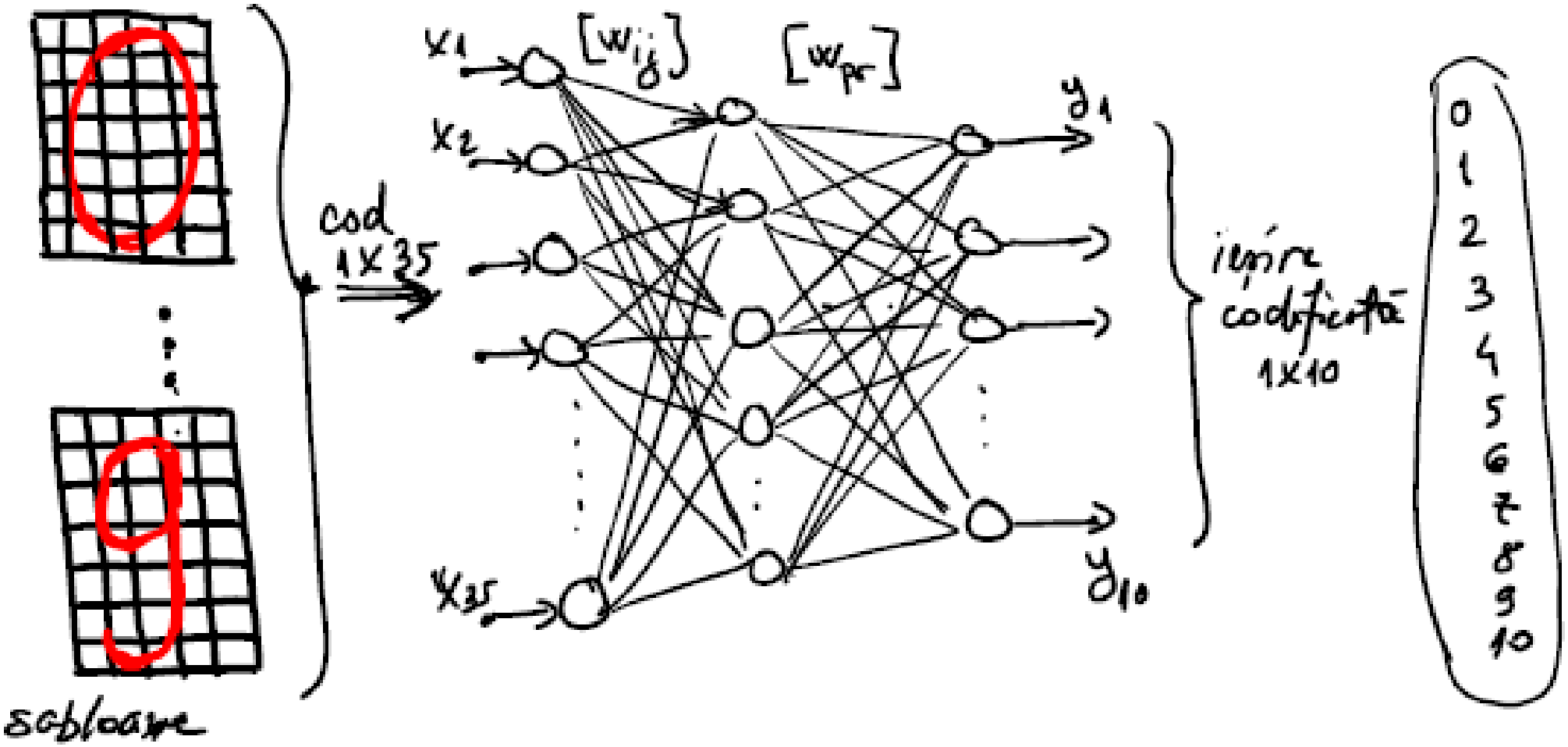
Notă: considerațiile de ordin practic se bazează pe toolboxul *nntool* din Matlab.

Utilizarea RNA multistrat

Principii

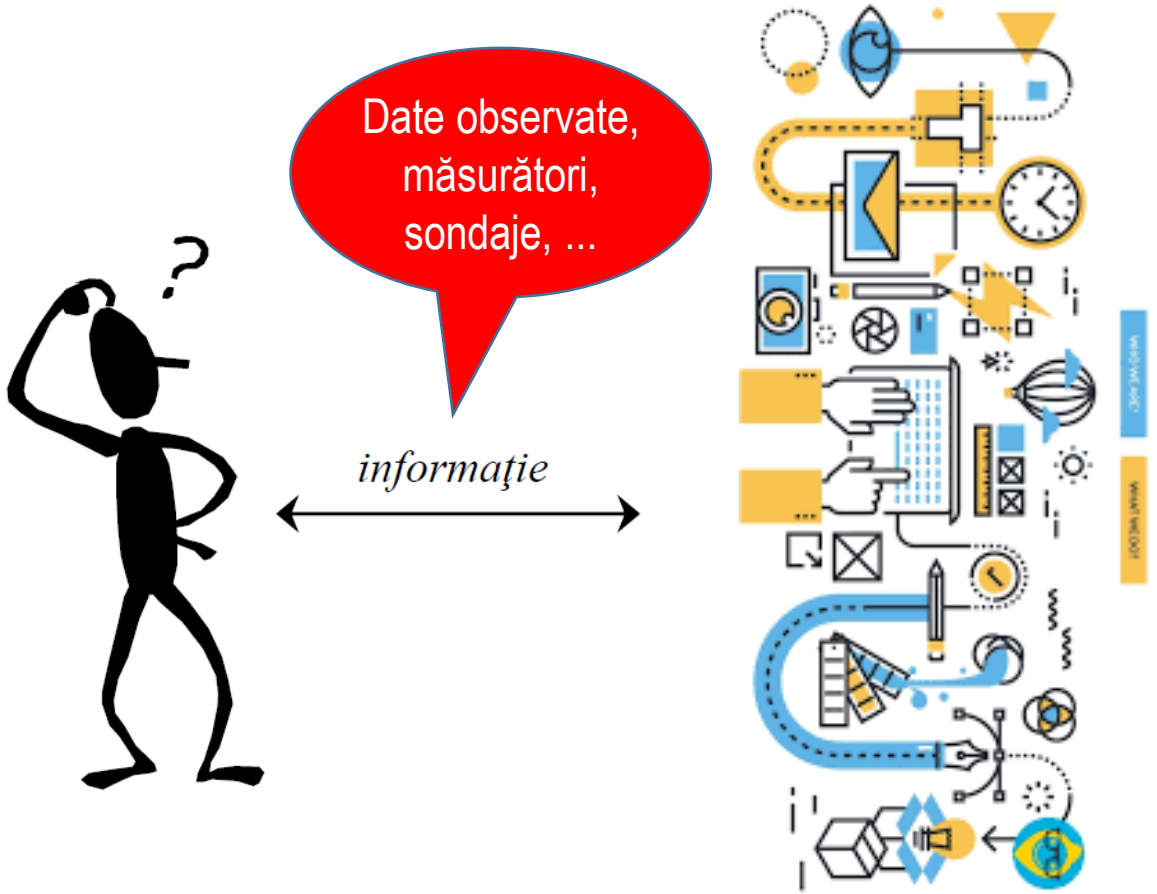
Recunoașterea formelor cu RNA




cod
1 x 35
x_1
x_2
x_{35}
$[w_{ij}]$
$[w_{jr}]$
y_1
y_{10}
ieșire
codificată
1 x 10
0
1
2
3
4
5
6
7
8
9
10
secvență

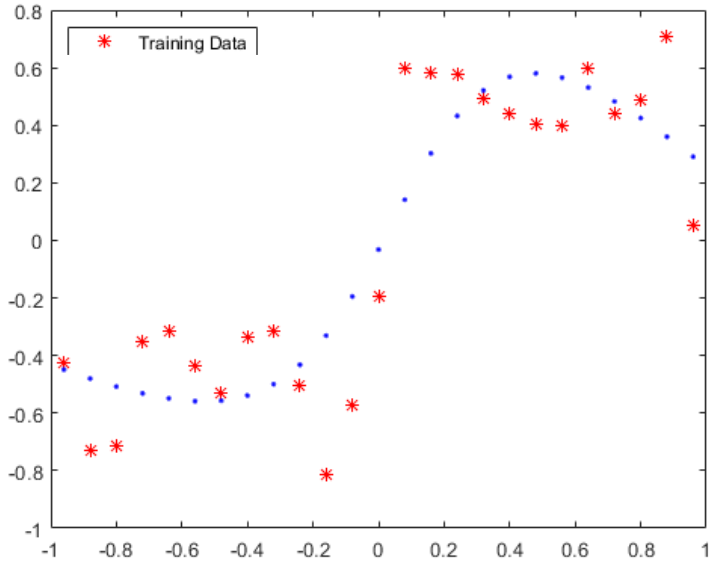
Aproximarea funcțiilor cu ajutorul RNA

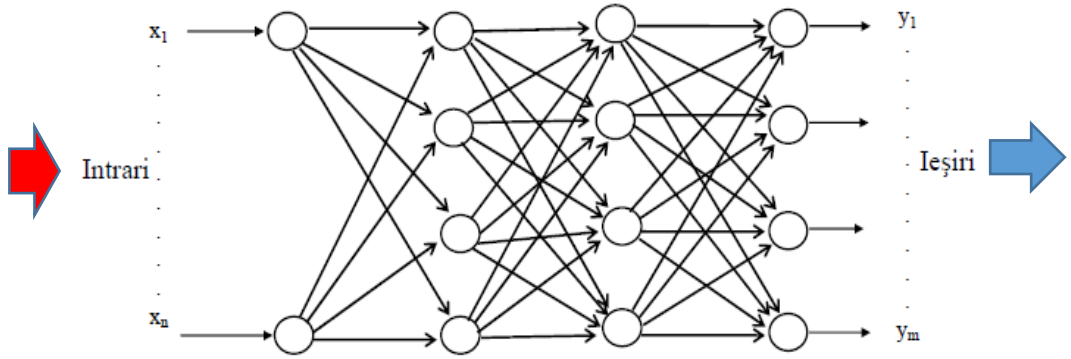
- Construirea ***modelelor de regresie neliniară*** pe baza datelor.
- Aplicații: modele inteligente pentru *identificarea* sistemelor/proceselor.

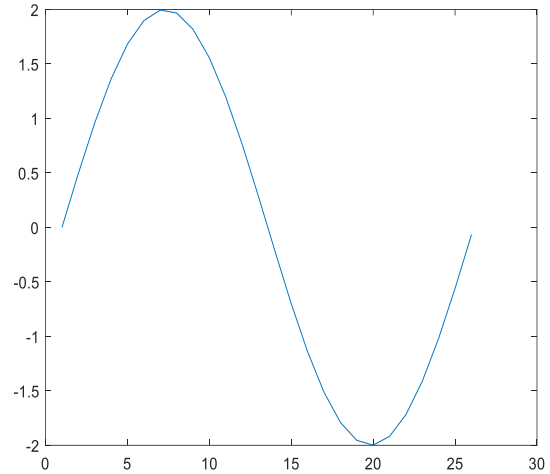


RNA aproximator universal

Identificarea sistemelor pe baza datelor de intrare-ieșire.







$Y = f(x1, x2, \ldots, xn)$

$[Y1, Y2, \ldots, Ym] = g(x1, x2, \ldots, xn)$

Procesul de antrenare (învățare) supervizată

- Procesul se derulează *iterativ* în cadrul unor *epoci de învățare pe baza celor doi pași ai învățării supervizate: foreward și backward*.
- *Iterație=parcurgerea pașilor foreward-backward pentru o (singură) pereche de antrenare șablon-ieșire țintă.*
- *Epocă de antrenare=parcurgerea câte unei iterații pentru fiecare pereche de antrenare.*

i. Pasul foreward: se prezintă o pereche de antrenare și se calculează *eroarea*.

ii. Pasul backward: se "*propagă eroarea înapoi*" process în care se ajustează ponderile prin legea de învățare.

iii. Se prezintă perechea de antrenare următoare și *se reia ciclul* pașilor *foreward-backward*.

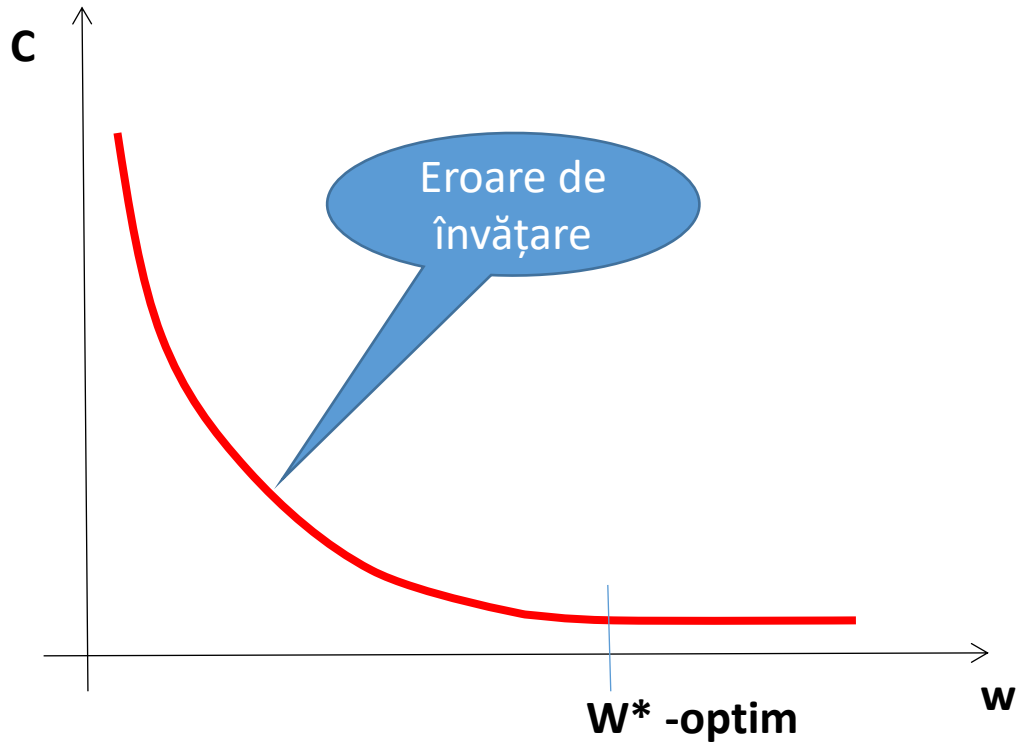
iv. La *epuizarea șabloanelor de învățare se înregistrează o epocă de antrenare a rețelei*

v. Se continuă cu *epoci succesive de antrenare* până când *eroarea globală* atinge un *minim absolut impus* sau până se atinge o *limită stabilită* pentru *numărul maxim de epoci* de antrenare.

Considerente de optimizare a procesului de antrenare

- Am arătat că o RNA feedforward poate fi antrenată pentru *aproximarea funcțiilor* (regresie neliniară) sau pentru *recunoașterea formelor*.
- În ambele situații procesul de instruire necesită *date* organizate în *perechi de antrenare* constituite ca un set de exemple (șabloane) de *răspuns corect* al rețelei - *intrări în rețea*, respectiv *ieșiri țintă corespondente*.
- **Scopul RNA feedforward** este de a realiza funcția ***F(w,x)*** notată cu $F_w(x)$ care aproximează (reproduce) funcția ***F(x)*** *cu erori minime* pentru toate exemplele posibile – atât cele învățate cât și altele diferite, asemănătoare din afara setului de antrenare.
- **Procesul de antrenare/învățare a RNA este esențial** și el ar trebui să decurgă optim în raport cu o **funcție de cost - C(w)**, numită în continuare *funcție de performanță.*

Învățare *optimă* = *minimizarea* funcției de cost



- Eroarea de aproximare a funcției date - ***F(x)*** de către funcția realizată de rețea - ***F(w,x)*** scade continuu până atinge un *minim* pentru *setul de valori optime* ale ponderilor (w*).
- În acest moment RNA se consideră *învățată* și *procesul de antrenare se oprește*.
- În structurile RNA învățarea este un *proces de optimizare*.

Metodologia de lucru cu RNA multistrat

Fluxul de lucru pentru procesul general de proiectare a rețelei neuronale are șapte pași principali:

1) Colectarea datelor
2) Crearea rețelei neuronale
3) Configurarea rețelei
4) Inițializarea ponderilor și a termenilor de nivel (bias)
5) Antrenarea rețelei
6) Validarea rețelei (analiză post-antrenare)
7) Utilizarea rețelei neuronale antrenate.

1: Colectarea și pregătirea datelor

- Premergător proiectării rețelei, trebuie colectate datele și pregătite eșantioanele (sabloanele) de antrenare. În general, este dificil să se încorporeze cunoștințe anterioare (apriorice) într-o rețea neuronală, prin urmare, precizia rețelei neuronale depinde de precizia și corectitudinea datelor utilizate pentru antrenarea rețelei.
- Este important ca datele să acopere *gama de intrări* pentru care rețeaua va fi utilizată. Rețelele multistrat pot fi instruite pentru a generaliza suficient de bine în gama de intrări pentru care au fost instruite. Cu toate acestea, ele nu au capacitatea de a extrapola cu precizie dincolo de acest interval, deci este important ca datele de instruire să se întindă pe întreaga gamă a spațiului de intrare.
- După ce datele au fost colectate, înainte ca acestea să fie utilizate pentru antrenarea rețelei trebuie efectuați următorii pași:
  - Preprocesarea datelor,
  - Împărțirea datelor în subseturi (pentru antrenare propriuzisă, testare, verificare)

Definirea subseturilor de date

Date de antrenare:

Acestea sunt prezentate rețelei *în timpul antrenamentului*, iar rețeaua este ajustată în funcție de eroarea răspunsurilor sale.

Date de validare:

Acestea sunt utilizate pentru a măsura capacitatea de ***generalizare a rețelei*** și pentru a opri instruirea atunci când generalizarea încetează să se îmbunătățească.

Date de testare:

Acestea nu au niciun efect asupra instruirii și, prin urmare, *oferă o măsură independentă a performanței rețelei* după antrenament.

2,3: Crearea și configurarea RNA

- După colectarea datelor, următorul pas este crearea **obiectului rețea**. Parametrii rețelei se vor *configura* în funcție de rolul RNA (aplicația propriuzisă) și de complexitatea problemei modelate.
- ***Configurarea*** este procesul de *setare a dimensiunilor* (numărul de neuroni) și intervalelor de intrare și de ieșire, setările de preprocesare ale intrărilor și ale ieșirilor (funcțiile de activare), urmat de inițializarea ponderilor. Astfel:

i. *Configurarea intrării RNA (numărul de neuroni de intrare)* se face la dimensiunea vectorului de trăsături ai formei de recunoscut sau la numărul parametrilor funcției de aproximat.

ii. *Configurarea ieșirii RNA (nr. de neuroni de ieșire)* se face la dimensiunea vectorului de date de ieșire țintă.

iii. *Configurarea numărului de straturi ascunse* și a numărului de neuroni ai acestora se face în funcție de complexitatea problemei de rezolvat. În general, straturile multiple de neuroni cu funcții de transfer neliniare permit rețelei să învețe relații neliniare între vectorii de intrare și ieșire. Rețelele feedforward cu mai multe straturi ar putea învăța relații complexe mai repede. Pentru majoritatea problemelor se recomandă să se înceapă cu două straturi și apoi să se crească la trei straturi, dacă performanța cu două straturi nu este satisfăcătoare.

iv. *Configurarea funcțiilor de preprocesare* se face prin *setarea tipurilor de funcții de activare* în funcție de natura problemei, astfel:

- pentru probleme de recunoaștere a formelor se adoptă funcții de activare *sigmoidale* (logsig, tansig),
- pentru probleme de aproximare a funcțiilor se alege funcția *liniară* (purelin).

4: Inițializarea ponderilor și a nivelelor (bias)

- Cea mai utilizată metodă de inițializare folosește generarea de numere aleatoare între -1 și 1.
- Se pot folosi și valori nule pentru inițializarea ponderilor în cazul RNA de tip perceptron.
- RNA poate fi antrenată *numai după aplicarea pasului de inițializare*.
- Ponderile pot fi *reinițializate*, dacă procesul de antrenare nu evoluează satisfăcător.

5: Antrenarea RNA – *funcția de performanță*

- Procesul de antrenare (învățare) a unei rețele neuronale implică ajustarea valorilor ponderilor și ale nivelurilor (bias) într-un mod *cât mai aproape de optim*, conform unei *funcții de performanță* a rețelei. Pentru rețelele feedforward *funcția de performanță implicită* este *eroarea pătratică medie* (mean squared error – *mse*) - dintre ieșirile curente ale rețelei (*a*) și ieșirile țintă (*t*):

$$F = mse = \frac{1}{N}\sum_{i=1}^{N}(e_i)^2 = \frac{1}{N}\sum_{i=1}^{N}(t_i - a_i)^2$$

- Altă funcție de performanță utilizată este *suma pătratelor erorilor* (sum of squared errors – *sse*):

$$\sum_{i=1}^{N}(t_i - a_i)^2$$

Antrenarea RNA (cont.) – *algoritmi optimi*

- ***Problema de bază a antrenării RNA*** *este căutarea minimului funcției de performanță (funcția de cost) în spațiul parametrilor rețelei (ponderile).*
- *Optimizarea antrenării*: orice algoritm standard de optimizare numerică poate fi utilizat pentru a optimiza *funcția de performanță – eroarea pătratică medie*,

Din *legea generală de învățare* se deduce că ajustarea ponderilor la fiecare iterație este direct proporțională cu rata de învățare (c), semnalul de învățare (r) (specific legii de învățare) și intrările curente în rețea (X):

$$\Delta W = W_{ij}(t+1) - W_{ij}(t) = X_j(t) \cdot r \cdot c \ ,$$

Considerând domeniul de variație al ponderilor *ca fiind continuu,* atunci evoluția ponderilor este caracterizată de un *gradient care determină performanța de antrenare*.

Gradientul depinde de produsul dintre intrarea curentă și semnalul de învățare, deci *depinde de legea de învățare*. Determinarea **direcției de căutare a gradientului negativ** face obiectul algoritmilor de optimizare.

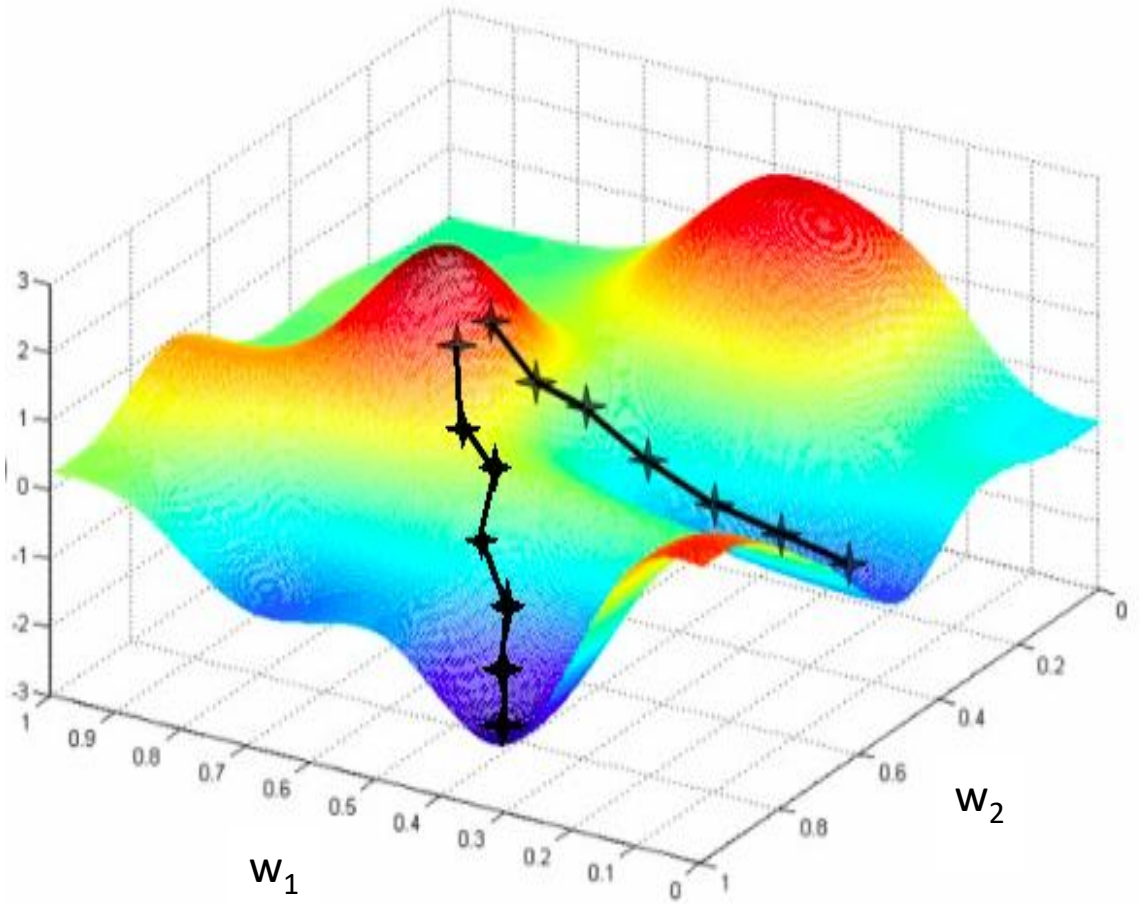
Conceptul de căutare a minimului funcției de cost/performanță

- Găsirea *setului de ponderi* care minimizează eroarea de antrenare, de exemplu *mse*
- Actualizarea ponderilor prin metoda ***gradient descent***:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$$

- În general căutarea minimului se face într-un spațiu multi-dimensional

(w1, w2, ...,wn)



Metode de căutare

- Metodele de căutare a minimului funcției de cost/performanță sunt deterministe sau stochastice (probabiliste). Cele deterministe pot fi:
  - de ordinul întâi – dacă utilizează derivatele de ordinul unu ale funcției de cost,
  - de ordinul doi – dacă utilizează derivate de ordinul doi ale funcției de cost,
- Există câteva metode care au demonstrat în practică performanțe excelente pentru antrenamentul rețelelor neuronale. Aceste metode de optimizare folosesc fie *gradientul performanței* rețelei în raport cu ponderile rețelei, fie *Jacobianul erorii rețelei* în raport cu ponderile.
- În continuare sunt prezentate cele mai utilizate metode ale algoritmilor de antrenare a RNA.

Algoritmi de antrenare a RNA

Algoritm/Metodă	Funcție Matlab
Levenberg-Marquardt	trainlm
Bayesian Regularization	trainbr
BFGS Quasi-Newton	trainbfg
Resilient Backpropagation	trainrp
Scaled Conjugate Gradient	trainscg
Conjugate Gradient with Powell/Beale Restarts	traincgb
Fletcher-Reeves Conjugate Gradient	traincgf
Polak-Ribiére Conjugate Gradient	traincgp
One Step Secant	trainoss
Variable Learning Rate Gradient Descent	traingdx
Gradient Descent with Momentum	traingdm
Gradient Descent	traingd

Cea mai rapidă funcție de antrenament este în general trainlm care este implicită pentru RNA feedforward. Metoda BFGS Quasi-Newton - trainbfg, este de asemenea destul de rapidă. Ambele metode tind să fie mai puțin eficiente pentru rețelele mari (cu mii de ponderi), deoarece *necesită mai multă memorie* și mai mult timp de calcul pentru aceste cazuri. De asemenea, trainlm funcționează mai bine pentru problemele de aproximare a funcților (regresie neliniară) decât la problemele de recunoaștere a formelor.

Atunci când se antrenează rețele mari și rețele de recunoaștere a formelor, trainscg și trainrp sunt alegeri bune. Cerințele de memorie sunt relativ mici și sunt mult mai rapide decât algoritmii standard de gradient descendent.

Antrenarea RNA (cont.) – Moduri de aplicare

Există două moduri distincte în care poate fi implementat antrenamentul RNA: modul *incremental* și modul *lot (batch)*:

- În modul *incremental*, este calculat *gradientul* și ponderile sunt actualizate după ce fiecare intrare (șablon de antrenare) este aplicată rețelei.
- În modul *batch*, ponderile se actualizează după ce toate intrările din setul de antrenament (șabloanele) au fost aplicate rețelei.
- Pentru majoritatea problemelor, de regulă, antrenamentul pe loturi este semnificativ *mai rapid* și produce *erori mai mici* decât antrenamentul incremental.

Algoritmi bazați pe metode deterministe de ordinul întâi

- Metodele deterministe de ordinul întâi folosesc numai valoarea gradientului local pentru aflarea direcției de minimizare. Se mai numesc metode de gradient negativ (‘gradient descent’ sau ‘steepest descent’).
- Constituie baza algoritmului de backpropagation (propagarea ‘inversă’ a erorii) specific RNA feedforward.

Algoritmul *Gradient Descent Backpropagation*

- Pentru a ilustrare modul în care funcționează instruirea, se ia în considerare cel mai simplu algoritm de optimizare – '*gradientul descendent' (gradientul negativ)*. Acesta actualizează ponderile și nivelurile (bias) la nivelul rețelei în direcția în care funcția de performanță *scade cel mai rapid*, adică *gradientul este negativ*.
- Minimizarea funcției de cost/performanță se face în lungul dreptei în direcția gradientului. Gradientul este reevaluat și procesul se repetă:

$$g_k = -\nabla C(wk)$$

- O iterație a acestui algoritm poate fi scrisă astfel:

$$w_{k+1} = w_k - \eta_k g_k$$

Unde: g_k este *gradientul* curent, iar η_k este *rata de învățare*. Această ecuație este iterată până când rețeaua converge.

- Direcțiile de căutare sunt succesiv perpendiculare (traiectorie de căutare în zig-zag).
- Algoritmul este rapid dar asigură o *convergență slabă* a antrenării rețelei și *se poate bloca pe minime locale*.

Parametrii algoritmului *Gradient Descent*

Ponderile și nivelurile sunt actualizate în direcția gradientului negativ al funcției de performanță. Algoritmul este controlat (inclusiv oprirea sa) prin următorii parametri:

- *Epochs* – numărul (maxim) de epoci (iterații) de antrenare prestabilit. Antrenamentul se oprește dacă numărul de iterații depășește epocile.
- *Goal* – Scopul/Obiectivul criteriului de oprire a procesului de antrenare. Antrenamentul se oprește dacă funcția de performanță scade sub valoarea obiectiv (un minim).
- *Timp* – Limita de timp pentru antrenare. Antrenamentul se oprește dacă timpul de antrenament este mai mare decât un anumit număr prestabilit de secunde.
- *min_grad* – Valoarea minimă a gradientului. Antrenamentul se oprește dacă magnitudinea gradientului este mai mică decât o valoare minimă prestabilită.
- *max_fail* – Numărul maxim de eșecuri succesive la căutarea minimului. Antrenamentul se oprește dacă performanța rețelei nu reușește să se îmbunătățească sau rămâne aceeași pentru *max_fail* epoci. Este asociat cu tehnica de oprire timpurie
- *Show* - Starea procesului de antrenare este afișată sau nu pentru fiecare iterație a algoritmului.
- *lr* – Rata de învățare multiplică negativul gradientului pentru a determina modificările la ponderi și niveluri. Cu cât rata de învățare este mai mare, cu atât este mai mare pasul de modificare. Dacă rata de învățare este prea mare, algoritmul devine instabil. Dacă rata de învățare este setată prea mică, algoritmul necesită mult timp pentru a converge.

Valori tipice ale parametrilor de antrenare prin metoda *Gradient Descent Backpropagation*

Exemplu de setare a parametrilor:

t.trainParam.epochs	1000	Maximum number of epochs to train
net.trainParam.goal	0	Performance goal
net.trainParam.lr	0.01	Learning rate
net.trainParam.max_fail	6	Maximum validation failures
net.trainParam.min_grad	1e-5	Minimum performance gradient
net.trainParam.time	inf	Maximum time to train in seconds
net.trainParam.show	25	Epochs between displays (NaN for no displays)
net.trainParam.showWindow	true	Show training GUI
net.trainParam.showCommandLine	false	Generate command-line output

Algoritmi bazați pe metode deterministe de ordinul doi

- Aceste metode utilizează și derivatele de ordinul doi ale funcției de cost/performanță C(w).
- Astfel de metode sunt:
  - metoda gradientului conjugat,
  - metoda Newton și
  - metodele de tip Quasi-Newton
- La aceste metode intervin structuri de calcul specifice: *Jacobianul erorii rețelei (matricea iacobiană a derivatelor de ordinul întâi) și matricea hessină a derivatelor de ordinul doi ale funcției de cost/performanță).*

Metode de gradient conjugat: varianta Powell-Beale

- Toți algoritmii de gradient conjugați încep la prima iterație cu căutarea în cea mai abruptă direcție de coborâre (negativă) a gradientului.
- Pentru toți algoritmii de *gradient conjugat*, direcția de căutare *este resetată periodic* la negativul gradientului. Momentul de resetare standard apare atunci când numărul de iterații devine egal cu numărul parametrilor de antrenat ai rețelei (ponderi și niveluri), Există însă și alte variante de metode de resetare care pot îmbunătăți eficiența instruirii, așa cum este metoda propusă de Powell pe baza unei versiuni anterioare propuse de Beale: aceasta resetează dacă între gradientul curent și gradientul anterior rămâne foarte puțină ortogonalitate. Acest lucru este testat cu următoarea inegalitate:

$$|\mathbf{g}^T_{k-1}\,\mathbf{g}_k| \geq 0.2\|\mathbf{g}_k\|^2$$

- Algoritmii de *gradient conjugat* sunt, de obicei, mult mai rapizi decât propagarea înapoi a ratei variabile de învățare. Algoritmii de gradient conjugat necesită doar puțin mai mult spațiu de stocare decât algoritmii mai simpli. Prin urmare, acești algoritmi sunt adecvați pentru rețelele cu un număr mare de ponderi.

Metode de gradient conjugat: varianta Fletcher-Reeves

Principiul de lucru:

- Toți algoritmii de gradient conjugat încep la prima iterație cu căutarea pe *cea mai abruptă direcție de coborâre* (negativă) a gradientului:

$$\mathbf{p}_0 = -\mathbf{g}_0$$

- Apoi este efectuată o *căutare liniară* pentru a determina *distanța optimă de deplasare* de-a lungul direcției curente de căutare:

$$\mathbf{w}_{k+1} = \mathbf{w}_k \eta_k \mathbf{p}_k$$

- Apoi, următoarea direcție de căutare este determinată astfel încât *să fie conjugată cu direcțiile de căutare anterioare*. Procedura generală pentru determinarea noii direcții de căutare este *combinarea celei mai abrupte direcții de coborâre cu direcția de căutare anterioară*:

$$\mathbf{p}_k = -\mathbf{g}_k + \beta_k \mathbf{p}_{k-1}$$

- Diferitele versiuni ale algoritmului de gradient conjugat se disting prin modul în care se calculează constanta β_k. De exemplu, pentru algoritmul de antrenare gradient conjugat în varianta Fletcher-Reeves procedura este:

$$\beta_k = (\mathbf{g}^T_k \mathbf{g}_k)/(\mathbf{g}^T_{k-1} \mathbf{g}_{k-1})$$

- Acesta este raportul dintre norma pătrată a gradientului curent și norma pătrată a gradientului anterior.

Metoda Newton
Metoda BFGS Quasi-Newton Backpropagation

- **Metoda lui Newton** originală este o alternativă la metodele de gradient conjugat pentru optimizare rapidă. Pasul de bază al metodei lui Newton este: $w_{k+1} = w_k - H_k^{-1} g_k$
- unde H_k^{-1} este matricea Hessian (derivate secundare) a *indicelui de performanță* la valorile curente ale ponderilor și nivelurilor(bias). Metoda lui Newton converge adesea mai rapid decât metodele de gradient conjugat. Din păcate, este complicat și costisitor să se calculeze matricea Hessian pentru rețelele neuronale avansate.

- ***Metodele cvasi-newtoniene*** (sau secante) sunt o clasă de algoritmi care se bazează pe metoda lui Newton, dar care nu necesită calculul derivatelor secundare. Acestea actualizează o *matrice Hessiană aproximativă* la fiecare iterație a algoritmului. Actualizarea este calculată în funcție de gradient. Metoda cvasi-Newton care a avut cel mai mare succes în literatura de specialitate este actualizarea Broyden, Fletcher, Goldfarb și Shanno (BFGS) consacrata sub denumirea de **BFGS Quasi-Newton Backpropagation**.

- Algoritmul BFGS necesită mai multe calcule în fiecare iterație și mai multă memorie decât metodele de gradient conjugat, deși converge, în general, în mai puține iterații. Hessianul aproximativ trebuie stocat, iar dimensiunea sa este n x n, unde n este egal cu numărul de ponderi și niveluri (bias) din rețea. Pentru rețele foarte mari, se recomandă utilizarea unuia dintre algoritmii de gradient conjugați. În schimb, pentru rețelele mai mici, metoda BFGS Quasi-Newton Backpropagation poate fi eficientă.

Algoritmul *Levenberg-Marquardt*

- La fel ca metodele cvasi-Newton, algoritmul Levenberg-Marquardt a fost conceput pentru a antrena RNA ținând cont de variațiile de ordinul doi ale funcției de cost/performanță, **dar evitând să se calculeze matricea Hessian**.

Principiul metodei:

- Când *funcția de performanță* are forma unei sume de pătrate (așa cum este tipic în antrenarea rețelelor de feed-forward), atunci matricea Hessian poate fi aproximată pe baza matricii jacobiene, astfel: $\boldsymbol{H = J^T J}$, iar gradientul $\boldsymbol{g = J^T e}$, unde $\boldsymbol{J}$ este matricea iacobiană care conține primele derivate ale erorilor de rețea în raport cu ponderile, iar $\boldsymbol{e}$ este un vector de erori ale rețelei. Avantajul este că matricea iacobiană poate fi calculată printr-o tehnică standard de propagare înapoi care este mult mai puțin complexă decât calculul matricei Hessian.
- Algoritmul Levenberg-Marquardt folosește aproximarea la matricea Hessian în următoarea actualizare de tip Newton:
  $\mathbf{w}_{k+1}=\mathbf{w}_k-[\mathbf{J}^T\mathbf{J}+\mu\mathbf{I}]^{-1}\mathbf{J}^T\mathbf{e}$
- **Parametrul scalar μ** este esential pentru funcționarea algoritmului Levenberg-Marquardt: când $\mu=0$, se reduce la o *metoda quasi Newton*, (folosind matricea Hessiană aproximativă). Când μ *este mare*, aceasta devine metoda clasică *gradient descent* cu o dimensiune mică a pasului. Metoda quasi Newton este mai rapidă și mai precisă, aproape de o eroare minimă, astfel încât scopul este de a trece la aceasta cât mai repede posibil. Astfel, μ este scăzut după fiecare pas de succes (reducerea funcției de performanță) și este crescut numai atunci când un pas provizoriu ar crește funcția de performanță. În acest fel, funcția de performanță este întotdeauna redusă la fiecare iterație a algoritmului.
- Levenberg-Marquardt se dovedește adesea ca fiind *cel mai rapid algoritm* de backpropagation și este recomandat ca **primă alegere** pentru antrenarea supervizată, deși necesită mai multă memorie decât alți algoritmi.
- Acest algoritm este cel mai potrivit pentru antrenarea rețelelor neuronale de dimensiuni moderate (până la câteva sute de ponderi).

Propagarea înapoi rezilientă

- Rețelele multistrat folosesc de obicei funcții de transfer sigmoidale în straturile ascunse. Aceste funcții sunt adesea numite funcții de „squashing”, deoarece produc efectul de *comprimare a* unui interval de intrare infinit într-un interval de ieșire finit.
- Funcțiile de activare sigmoidale se caracterizează prin faptul că *pantele lor se apropie de zero pe măsură ce intrarea devine mare*. Acest lucru cauzează o problemă atunci când se utilizează cea mai abruptă descendență în procesul de antrenare a rețelei, deoarece gradientul poate avea o magnitudine foarte mică determinând astfel *mici modificări ale ponderilor și nivelurilor*, chiar dacă acestea sunt sunt încă departe de valorile optime.
- **Scopul algoritmului de antrenare rezilient backpropagation** este de a elimina efectul dăunător indus de mărimea derivatelor parțiale. Astfel, *numai semnul derivatei poate determina direcția actualizării ponderii; magnitudinea derivatei neavând niciun efect asupra actualizării ponderii*. Ponderile sunt ajustate în funcție astfel:

$$\Delta w_{k+1} = \Delta w_k \, \text{sign}(g_k)$$

- **Mărimea modificării ponderii** este determinată de o valoare separată de actualizare. Valoarea de actualizare pentru fiecare pondere și nivel este mărită cu un factor (*delt_inc*) ori de câte ori derivata funcției de performanță în raport cu ponderea respectivă are același semn pentru două iterații succesive. Ea va fi redusă cu un factor (*delt_dec*) ori de câte ori derivata în raport cu ponderea respectivă schimbă semnul față de iterația anterioară. Dacă derivata este zero, valoarea de actualizare rămâne aceeași. Dacă ponderile oscilează, ajustarea acestora este redusă. Dacă ponderea continuă să se schimbe în aceeași direcție pentru mai multe iterații, magnitudinea schimbării crește.

Studiu de caz

- Se prezintă metodologia de lucru cu RNA printr-un exemplu implementat în Matlab cu funcția *nntool*.

NOTA:

- *nntool* este funcțională în versiunile Matlab până la R2021b.
- *nnstart* mai poate fi folosit în versiunile până la R2025b.
- Începând cu versiunile R2026, se folosesc exclusiv aplicațiile înlocuitoare asfel:

Pentru ce foloseai nntool/nnstart	**Înlocuitorul modern în MATLAB R2025b**
Clasificare (Pattern Recognition)	Classification Learner App
Regresie / Ajustare date (Fitting)	Regression Learner App
Grupare date (Clustering)	Cluster Data Live Editor Task
Deep Learning / Rțele complexe	Deep Network Designer

nntool: interfață grafică (GUI) pentru import, creare, utilizare și export al rețelelor neuronale și a datelor

Pasi de lucru:

1. Importă datele de intrare și țintă din spațiul de lucru cu [Import].

2. Crează o rețea nouă cu [New].

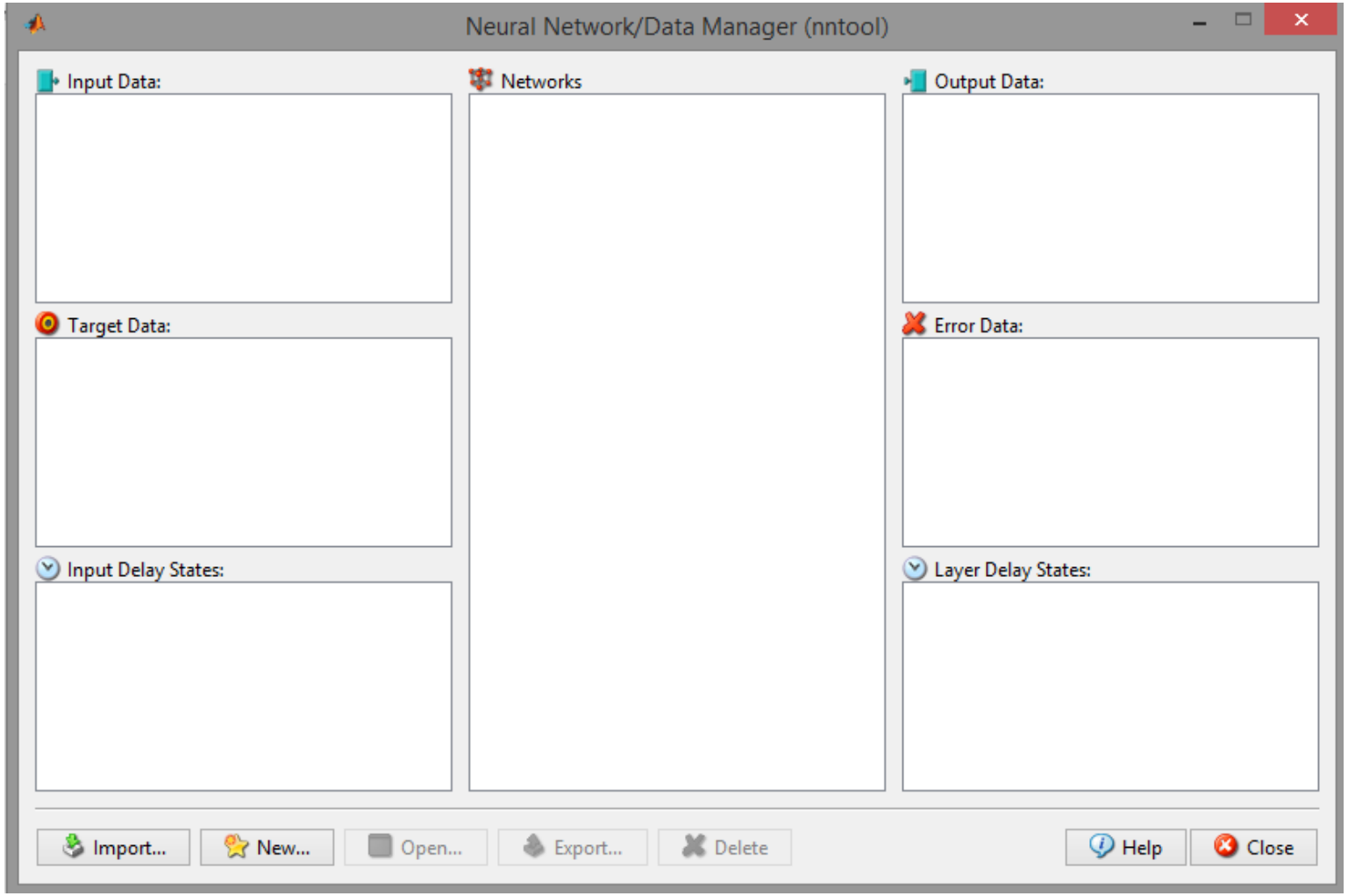
3. Selectează o rețea din lista Rețele și clic pe [Open]

4. Selectează pentru instruire rețeaua creată sau selectată în fereastra de rețea.

5. Selectează datele de intrare și țintă și clic pe [Train].

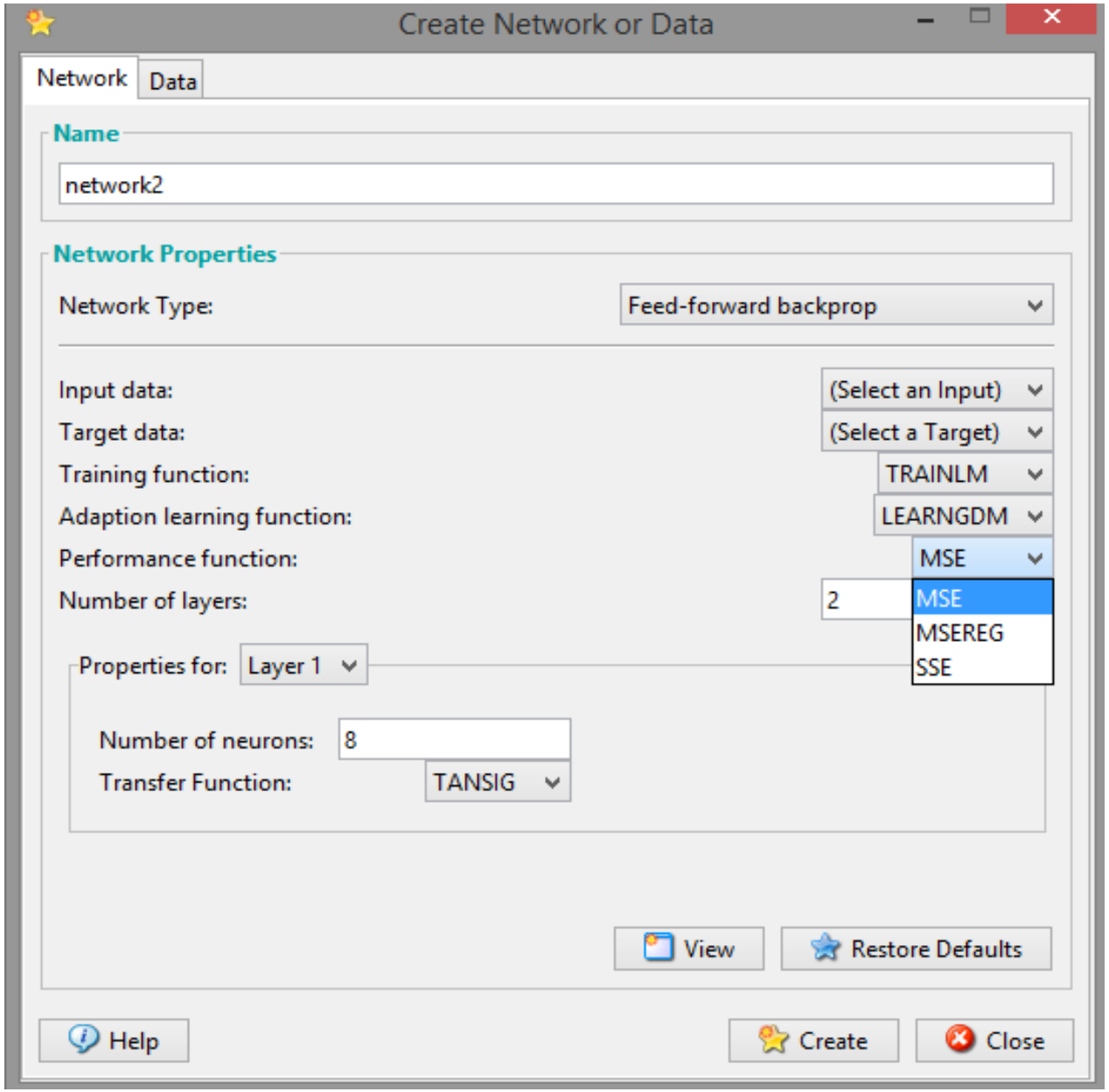
6. De asemenea, se poate utiliza fereastra de rețea pentru a simula rețeaua sau

7. Efectuarea altor sarcini precum, vizualizarea reinitializarea și editarea ponderilor.



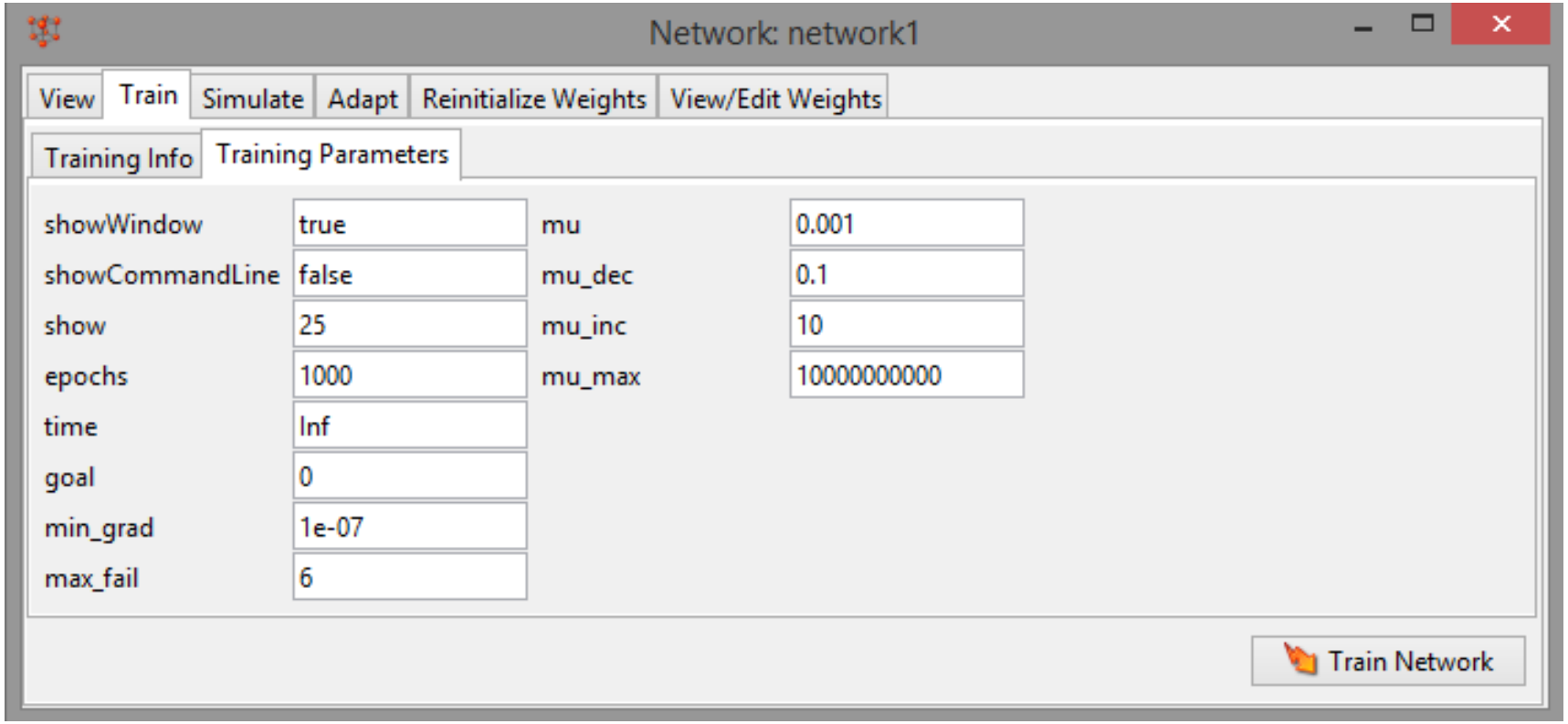
Configurarea structurii rețelei

- **Network Type**: selectează tipul de rețea
- **Training function**: selectează metoda/algoritmul de optimizare a funcției de cost/performanță în procesul de antrenare (de ex. TRAINLM – algoritmul Levenberg-Marquardt backpropagation).
- **Adaption function**: selectează funcția de învățare pentru adaptarea ponderilor.
- **Performance function**: selectează funcția de cost/performanță (de ex. mse- mean squared error).
- **Transfer function**: funcția de activare utilizată pentru neuronii unui anumit strat



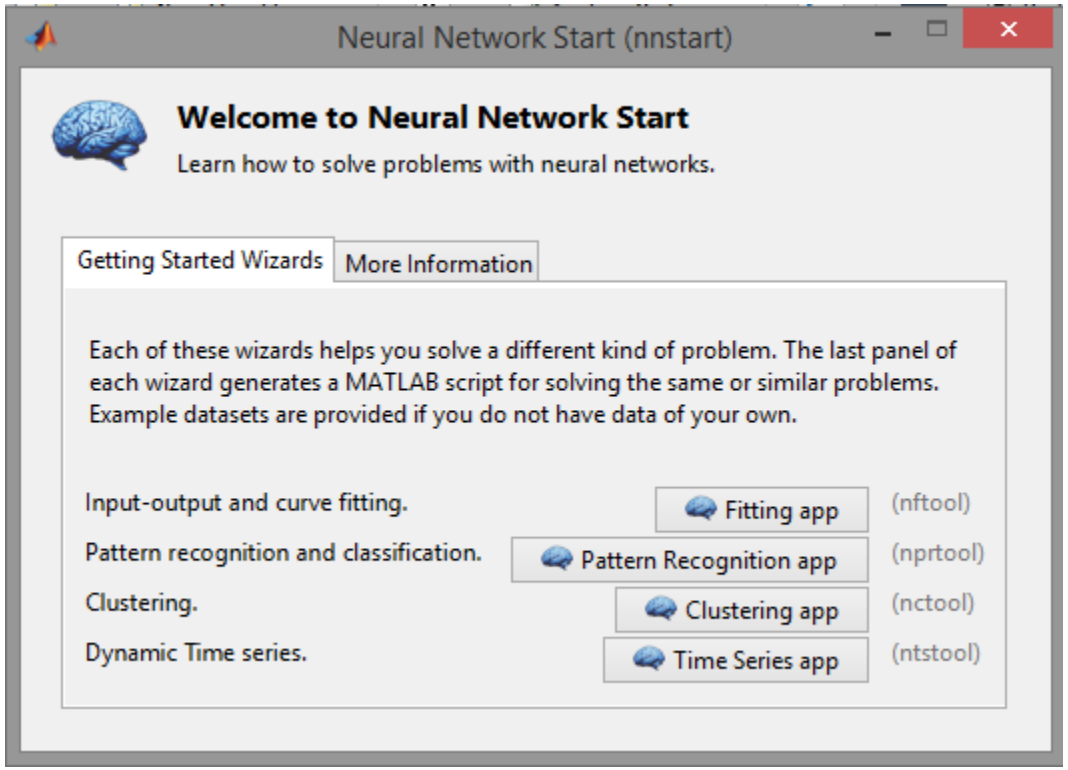
Configurarea parametrilor de antrenare

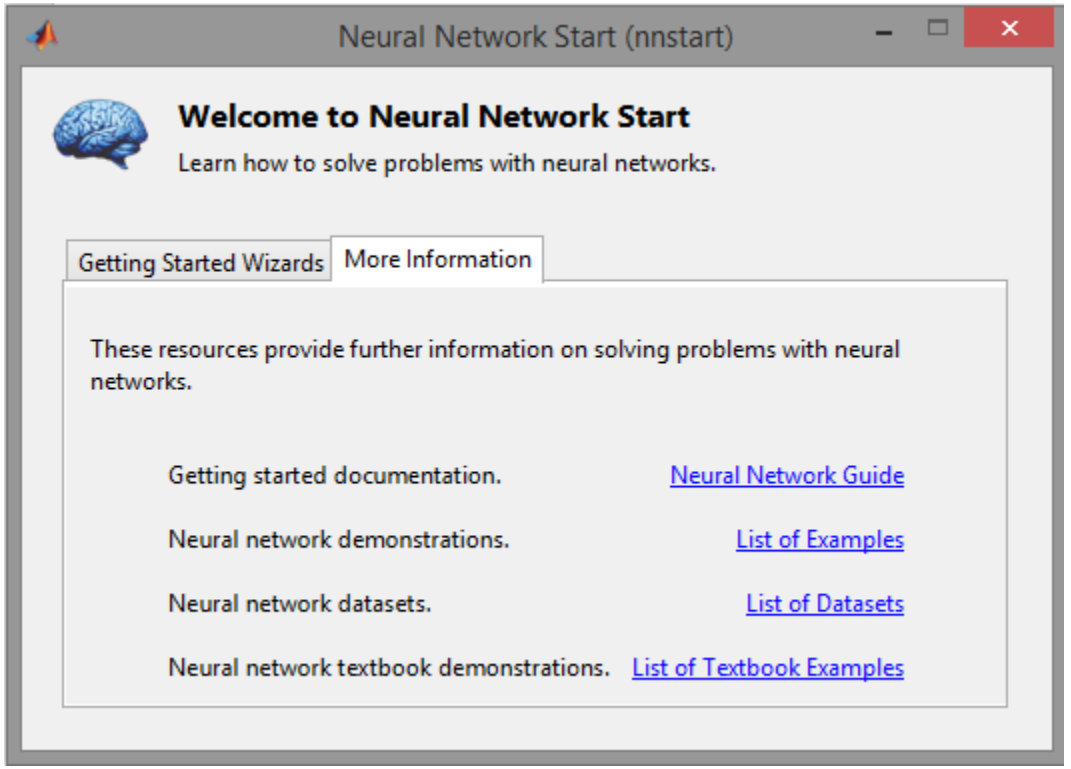
- Conține parametrii generali și particulari în funcție de tipul algoritmului de antrenare
- De exemplu, pentru **Training function** setată cu algoritmul *Levenberg-Marquardt* backpropagation (funcția mai performantă TRAINLM) parametrii impliciți sunt următorii:



Funcția nnstart

Funcția deschide o interfață grafică pentru configurarea și antrenarea RNA pentru aplicații de *aproximare a funcțiilor(modelelor)*, *recunoașterea formelor*, *clasificare-grupare* și *estimări pentru serii temporale*. De asemenea, oferă acces la seturi de date, exemple și informații utile pentru lucrul cu RNA.





Concluzii

1. RNA multistrat pot fi utilizate atât pentru *aproximarea funcțiilor*, cât și pentru problemele de *recunoaștere a formelor* folosind metoda de învățare supervizată.
2. Procesul de instruire necesită *date* organizate în *perechi de antrenare* constituite ca un set de exemple (șabloane) de *răspuns corect* al rețelei - *intrări în rețea*, respectiv *ieșiri țintă corespondente*.
3. Procesul de antrenare/învățare a RNA este esențial și el ar trebui să decurgă optim în raport cu o funcție de cost - C(w), numită *funcție de performanță.*
4. *Problema de bază a antrenării RNA este căutarea minimului funcției de performanță (funcția de cost) în spațiul parametrilor rețelei (ponderile).*
5. Există câteva metode care au demonstrat în practică performanțe excelente pentru antrenamentul rețelelor neuronale. Aceste metode de optimizare folosesc fie *gradientul performanței* rețelei în raport cu ponderile rețelei, fie *Jacobianul erorii rețelei* în raport cu ponderile.
6. *Metodologia de lucru cu RNA multistrat conține 7 pași*: (1) Colectarea datelor, (2) Crearea rețelei neuronale, (3) Configurarea rețelei, (4) Inițializarea ponderilor și a termenilor de nivel (bias), (5) Antrenarea rețelei, (6) Validarea rețelei (analiză post-antrenare), (7) Utilizarea rețelei neuronale antrenate.